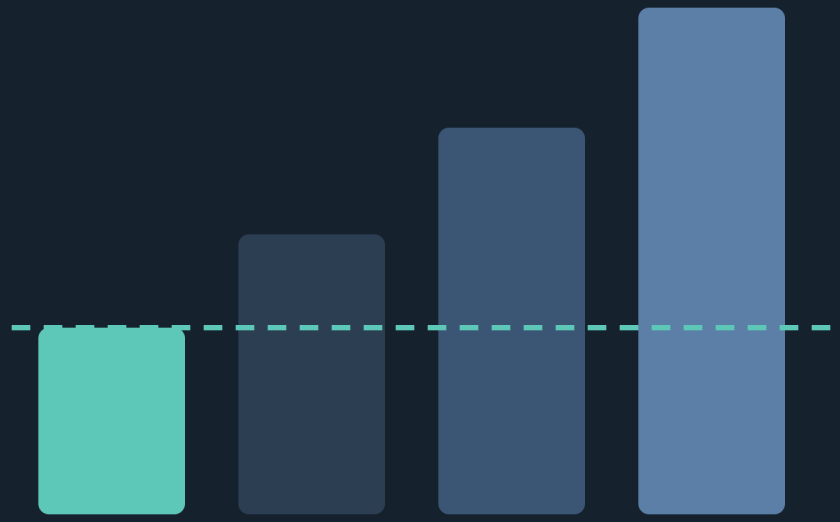


COW 데이터사이언스 · 5주차 이론

베이스라인 모델 구축

머신러닝·딥러닝 실험 설계에서 비교 기준을 세우는 방법

발표 · 김택동

베이스라인 모델의 정의

이후 모든 모델의 성능을 해석하기 위한 비교 기준(reference point)으로, 최소한의 가정과 설정으로 구축한 모델

Lower bound

성능 하한선

문제에서 최소한 달성해야 할 성능 수준. 이보다 낮으면 모델이 데이터의 유의미한 패턴을 학습하지 못한 것으로 판단한다.

Relative reference

상대적 해석

성능은 절대값보다 기준 대비 개선 폭(Δ)으로 해석한다. 같은 F1 0.80도 기준이 0.78인지 0.50인지에 따라 의미가 다르다.

Reproducibility

재현성

기본 하이퍼파라미터, 고정된 데이터 분할과 랜덤 시드로 구성해 이후 실험과 동일한 조건에서 비교한다.

02 · 목적

베이스라인 구축의 목적

정량적 검증

특성 공학, 모델 변경, 튜닝의 효과를 동일 조건에서 측정한 점수 차이(Δ)로 검증한다.

파이프라인 검증

데이터 로딩 → 전처리 → 학습 → 평가 전 과정을 가벼운 모델로 먼저 실행해 라벨 불일치, 형상 오류 등 구현 결함을 조기에 발견한다.

난이도·품질 진단

단순 모델이 높은 성능을 보이면 비교적 쉬운 문제이고, 더미 수준에 머물면 특성 정보 부족이나 라벨 노이즈를 우선 검토한다.

오컴의 면도날(Occam’s razor) 복잡한 모델은 단순한 모델보다 의미 있게 우수할 때에만 정당화된다

03 · 구성

베이스라인의 계층 구조

STEP 1 · 하한선

더미 모델

입력 특성을 사용하지 않는 규칙 기반 예측. 분류는 최빈값·클래스 비율, 회귀는 평균·중앙값

STEP 2

선형 모델

로지스틱 회귀, 선형 회귀(Ridge). 선형 관계를 가정하며 계수로 변수의 영향을 해석할 수 있다. 특성 스케일링 필요

STEP 3

트리 앙상블

랜덤포레스트(배깅), XGBoost, LightGBM (부스팅). 비선형 관계와 특성 간 상호작용을 포착하며 스케일링이 불필요하다

STEP 4

딥러닝

CNN (이미지), RNN, Transformer (시퀀스·텍스트). 원시 데이터에서 표현을 스스로 학습하나 데이터 요구량과 연산 비용이 크다

← 단순성 · 해석 가능성 · 학습 속도

표현력 · 데이터 요구량 · 연산 비용 →

데이터 유형별 베이스라인

데이터	1차 베이스라인	2차 베이스라인	선택 근거
정형 · 분류	로지스틱 회귀	랜덤포레스트, LightGBM	정형 데이터에서는 트리 기반 모델이 딥러닝보다 우수하거나 대등한 경우가 많음 [1]
정형 · 회귀	선형 회귀(Ridge)	랜덤포레스트, LightGBM	선형 관계의 설명력을 먼저 확인한 뒤 비선형 모델과 비교
이미지	ImageNet 사전학습 CNN(ResNet 등)의 선형 탐색	전체 미세조정 (fine-tuning)	사전학습 표현의 전이로 적은 데이터에서도 높은 성능
텍스트	TF-IDF + 로지스틱 회귀	사전학습 언어모델(BERT 등) 미세조정	단어 빈도 기반 표현만으로도 빠르고 강한 기준선 형성
시계열	나이브 예측(직전 값), 계절 나이브	이동평균, LightGBM (시차 특성)	나이브 예측을 넘지 못하면 예측력이 없는 것으로 판단

[1] Grinsztajn, Oyallon, Varoquaux (2022). Why do tree-based models still outperform deep learning on tabular data? NeurIPS 2022 Datasets and Benchmarks Track.

베이스라인 구축 절차

1

데이터 분할

학습·검증·테스트 분리. 분류는 층화 추출, 시계열은 시간 순서 유지, random_state 고정

→

2

평가 지표 선정

분류: Accuracy, F1, ROC-AUC. 회귀: MAE, RMSE. 문제 목적과 클래스 불균형을 고려

→

3

더미 모델 평가

Dummy Classifier · Regressor로 성능 하한선 산출

→

4

단순 모델 학습

기본 하이퍼파라미터 사용. 전처리는 Pipeline으로 묶어 데이터 누수 방지

→

5

결과 기록

모델, 전처리, 지표, 점수, 시드를 실험 기록표로 관리

원칙 이 단계의 목적은 성능 극대화가 아니라, 전 과정을 재현 가능한 형태로 한 번 끝까지 실행(end-to-end run)하는 것이다

구현 예시 (scikit-learn)

```python
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold, cross_val_score

models = {
    "Dummy": DummyClassifier(strategy="most_frequent"),
    "LogReg": make_pipeline(StandardScaler(),
                            LogisticRegression(max_iter=1000)),
    "RF": RandomForestClassifier(random_state=42),
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in models.items():
    s = cross_val_score(model, X_train, y_train, cv=cv, scoring="f1_macro")
    print(f"{name}: {s.mean():.3f} ± {s.std():.3f}")
```

성능 하한선

most_frequent 전략은 최빈 클래스만 예측해 클래스 불균형 정도를 그대로 드러낸다

누수 방지

스케일러를 Pipeline 안에 두어 각 폴드의 학습 데이터로만 fit한다

평균 ± 표준편차

5겹 층화 교차검증으로 분할에 따른 우연한 차이와 실제 성능 차이를 구분한다

07 · 유의사항

베이스라인 구축 시 유의사항

튜닝 배제

베이스라인 단계에서 하이퍼파라미터를 조정하면 이후 개선이 모델 구조 때문인지 튜닝 때문인지 분리할 수 없다. 기본값으로 고정한다.

동일한 실험 조건

데이터 분할, 전처리, 평가 지표, 랜덤 시드를 모든 모델에 동일하게 적용해야 성능 차이를 모델의 차이로 해석할 수 있다.

정확도 역설 (Accuracy paradox)

양성 비율이 10%인 데이터에서는 더미 모델도 정확도 90%를 얻는다. 불균형 데이터에서는 F1, PR-AUC를 함께 보고한다.

데이터 누수 (Data leakage)

스케일링·결측치 대체를 전체 데이터로 fit하거나 미래 정보가 특성에 포함되면 성능이 과대평가된다. 테스트 데이터는 최종 평가에만 사용한다.

요약

베이스라인은 실험의 비교 기준이며, 모든 개선은 이 기준 대비 차이로 평가한다

더미 모델 → 선형 모델 → 트리 앙상블 → 딥러닝

- 단순성과 재현성을 갖춘 모델로 성능 하한선을 설정한다
- 동일한 분할·지표·시드 아래에서 복잡도를 단계적으로 높인다
- 결과를 기록해 이후 학습·성능 향상 실험의 기준으로 사용한다

다음 주제 · 6주차 모델 학습 및 성능 평가 방법